VITAL Workshop Tutorials

Finbar Argus and Beatrice Ghitti

March 2025

0 Software Setup

For setting up software, please follow the instructions detailed here: `https://finbarargus.github.io/circulatory_autogen/getting-started/`

1 0D Bond Graph Blood Flow Tutorial

Follow the demonstration on the whiteboard. Then complete the following tasks:

1. Write out the bond-graph diagram for a pressure inlet, pressure outlet vessel.
2. Derive the system of equations for the vessel. Knowns:
   - $C = 1 \cdot 10^{-8}\,\mathrm{m^3 \cdot Pa^{-1}}$
   - $R_1 = 1 \cdot 10^{5}\,\mathrm{Pa \cdot s \cdot m^{-3}}$
   - $R_2 = 2 \cdot 10^{5}\,\mathrm{Pa \cdot s \cdot m^{-3}}$
   - $I_1 = 10\,\mathrm{Pa \cdot s^2 \cdot m^{-3}}$
   - $I_2 = 10\,\mathrm{Pa \cdot s^2 \cdot m^{-3}}$
   - $P_{in} = \left[1.2 \cdot 10^{4} + 2 \cdot 10^{3} \sin(2\pi t)\right]\,\mathrm{Pa}$
   - $P_{out} = 1 \cdot 10^{3}\,\mathrm{Pa}$
3. Implement the equations in OpenCOR.
4. Plot the pressure at the centre of the vessel and the flow at the inlet.
5. Extra: Swap the tube law with the following:

$$P(t) = P_0 + K\left[\left(\frac{q(t)}{q_0}\right)^m - \left(\frac{q(t)}{q_0}\right)^n\right] + P_{ext} \tag{1}$$

   - Use the following knowns:
     - $K = 5.3 \cdot 10^{4}\,\mathrm{Pa}$
     - $q_0 = 1 \cdot 10^{-4}\,\mathrm{m^3}$
     - $m = 0.5$
     - $n = 0$
     - $P_0 = 0\,\mathrm{Pa}$
     - $P_{ext} = 0\,\mathrm{Pa}$
   - Discuss whether (P_0, q_0) should be calibrated from data, or $P_0 = 0$ and $q_0 = q_{us}$ should be used. Simulate the differences in OpenCOR.

2 Circulatory System Generation Tutorial

1. Navigate to the `<path_to_trainingschool01>/20250318-day01/simple_physiological_nonlinear_visco` directory. We will call this directory `{CA_user_dir}` and the circulatory Autogen directory `{project_dir}`.
   - The `{CA_user_dir}/resources` directory contains input files that define the modules that are used in your models and their parameters (and later the information for parameter identification).

- The `{project_dir}` contains src files for Circulatory Autogen and configuration files for modules (defining the mathematics and the allowable connections for the modules).

2. Open `{project_dir}/user_run_files/user_inputs.yaml` and set

   `user_inputs_path_override: {CA_user_dir}/simple_physiological_user_inputs.yaml`

   Important: For Windows, make sure you use "C:" at the start of the directory and backslash instead of forward-slash.

   This lets CA know where to look for config files for the model that you want to generate and eventually calibrate. The files that are needed to generate the circulatory system model considered in this tutorial are:

   - `{CA_user_dir}/resources/simple_physiological_nonlinear_visco_vessel_array.csv` : This file defines the modules that are used and how they are connected together. Figure 1 shows the model that we will first generate.

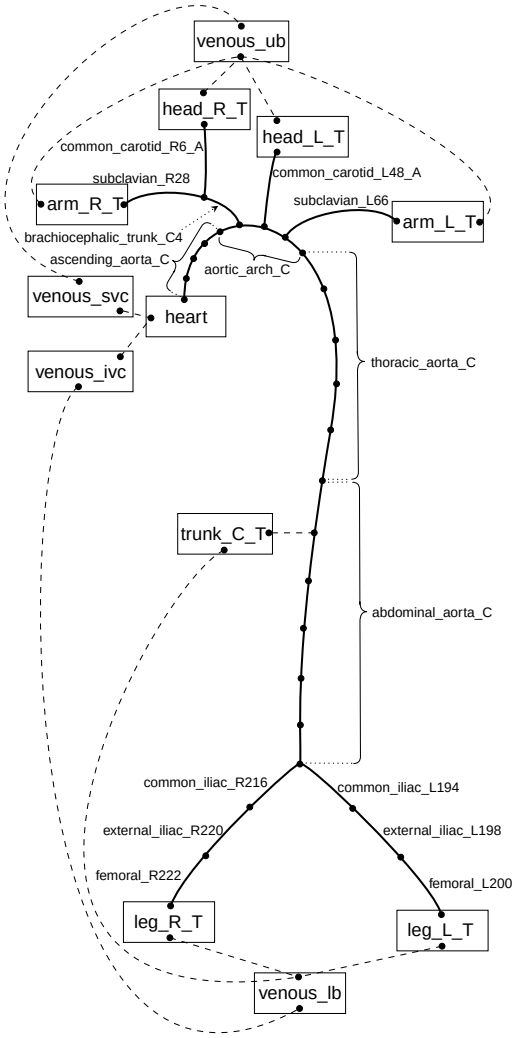


Figure 1: Schematic of the simple physiological CVS model.

   - `{Ca_user_dir}/resources/simple_physiological_nonlinear_visco_parameters.csv`: This file defines the constants and boundary conditions needed to simulate the model.
   - `{Ca_user_dir}/resources/simple_physiological_nonlinear_visco_params_for_id.csv`: This file defines the parameters that will be identified in the calibration steps (not modified in this tutorial but explained in `https://finbarargus.github.io/circulatory_autogen/parameter-identification/`).

- `{Ca_user_dir}/resources/simple_physiological_nonlinear_visco_obs_data.json`: This file defines the experimental/clinical protocols and the ground truth values that you want your outputs to replicate (also not modified in this tutorial).
- `{Ca_user_dir}/simple_physiological_user_inputs.yaml`: This is the main configuration file for your model, it specifies the location of your input files, the solver info, the parameter identification algorithm info, and more (you shouldn't need to edit it for this tutorial).

3. Navigate to `{project_dir}/user_run_files/` and execute the following command to generate the model:

   `./run_autogeneration.sh`

   The model will be generated in

   `{CA_user_dir}/generated_models/simple_physiological_nonlinear_visco`

4. Open the generated model (`simple_physiological_nonlinear_visco.cellml`) in OpenCOR and plot the left subclavian pressure: `subclavian_L66/u`.

5. Now, assuming that we want to use PPG measurements from the finger to calibrate the model, we want to extend the vasculature network so that it includes arteries down to the fingers. The network we will include (for the left arm only) is displayed in Figure 2.

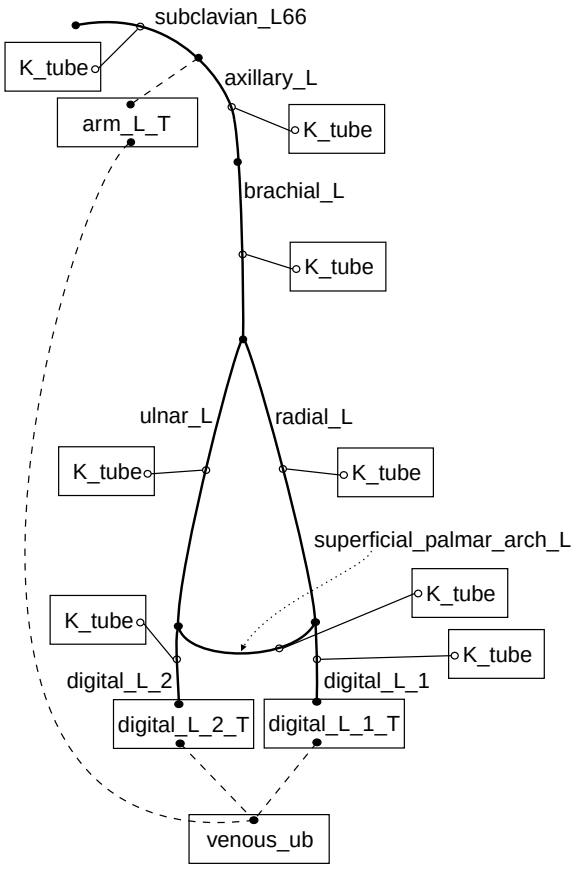


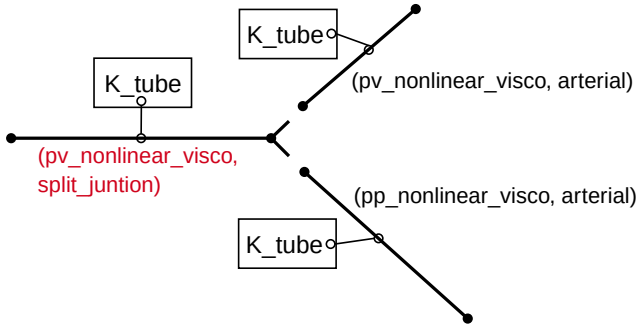
Figure 2: Schematic of the network that you need to include in the circulatory system model.

6. Modify the `{Ca_user_dir}/resources/simple_physiological_nonlinear_visco_vessel_array.csv` and `{Ca_user_dir}/resources/simple_physiological_nonlinear_visco_parameters.csv` files so that they include the vessels in Figure 2.

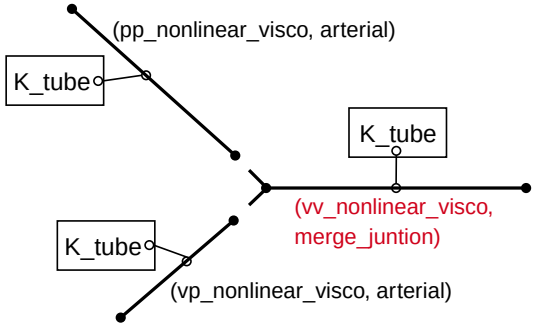
   - All parameters that need to be included in the model are given in the following file:

`{Ca_user_dir}/resources/extra_parameters_LeftArm.csv`

which you can copy and paste into the parameters file. Note: when extending the left arm arterial network, you also have to modify the parameters of the left subclavian artery (specifically, its length `l_subclavian_L66` and reference radius `r_0_subclavian_L66`).



(a) Example of vessel modules to use to define a **bifurcation**.



(b) Example of vessel modules to use to define a **merge junction**.

Figure 3: Schematic of some vessel modules to model junctions. Each vessel module is defined as the pair "(BC_type, vessel_type)", and it is also coupled to a "K_tube" module to compute its material properties (e.g., the stiffness parameter K in the nonlinear elastic tube law (1)).

- You will need to use a combination of "arterial", "split_junction" (1-in-2-out) and "merge_junction" (2-in-1-out) to model the superficial palmar arch loop (see Figure 3).

- BC types (Boundary condition types) must be compatible, an output flow v (e.g., pv_nonlinear_visco) must connect to an inlet pressure p (e.g., pp_nonlinear_visco or pv_nonlinear_visco) and vice versa. Use "nonlinear_visco" for all, to use vessel modules with nonlinear viscoelastic tube law.

- Currently, we enforce that the split/merge side of a vessel needs to have a flow boundary condition; therefore, merge_junction must be vv or vp type, while split_junction must be vv or pv type.

- Re-calculate estimates of terminal resistance R_T for { arm_L_T, digital_L_1_T, digital_L_2_T } according to the following formula

$$R_{T_j} = \frac{P_{art} - P_{ven}}{CO * BFF_j}. \tag{2}$$

  Assume that the desired cardiac output in the model is $CO = 100 \cdot 10^{-6}\,\mathrm{m}^3/\mathrm{s}$, estimated mean arterial and venous pressures are $P_{art} = 12\,000\,\mathrm{Pa}$ and $P_{ven} = 666\,\mathrm{Pa}$, respectively, and the desired fractional flows to the three terminals are:

Terminal	BFF (%)	C_T ($m^3 \cdot Pa^{-1}$)
arm_L_T	1.75%	$1 \cdot 10^{-10}$
digital_L_1_T	0.875%	$1 \cdot 10^{-11}$
digital_L_2_T	0.875%	$1 \cdot 10^{-11}$

Table 1

Set the terminal compliance C_T to be small for now (see Table 1). These parameters could be further calibrated to PPG data.

7. Again, navigate to `{project_dir}/user_run_files/` and do the following command to generate the model with your new arm circulation connected:

   ```
   ./run_autogeneration.sh
   ```

8. Open the generated model in OpenCOR and plot the digital_L_2 pressure: `digital_L_2/u`.

9. Extra: Navigate to `{project_dir}/user_run_files/` and do the following command to run a parameter identification algorithm on the generated model:

   ```
   ./run_param_id.sh {number_of_cores}
   ```

   This will run the optimisation algorithm defined in

   ```
   {CA_user_dir}/simple_physiological_user_inputs.yaml
   ```

   to calibrate the parameters defined in

   ```
   {Ca_user_dir}/resources/simple_physiological_nonlinear_visco_params_for_id.csv
   ```

   to the ground truth values defined in

   ```
   {Ca_user_dir}/resources/simple_physiological_nonlinear_visco_obs_data.json
   ```

   You can then run the following command to plot and compare the model outputs with the ground truth observables:

   ```
   ./plot_param_id.sh
   ```

   The plotting can be run while the parameter i.d. is running, as soon as one generation is finished. All identified parameter information and plots are stored in

   ```
   {Ca_user_dir}/param_id_output/
   ```

   For more information on using Circulatory Autogen for parameter identification, see `https://finbarargus.github.io/circulatory_autogen/parameter-identification/`

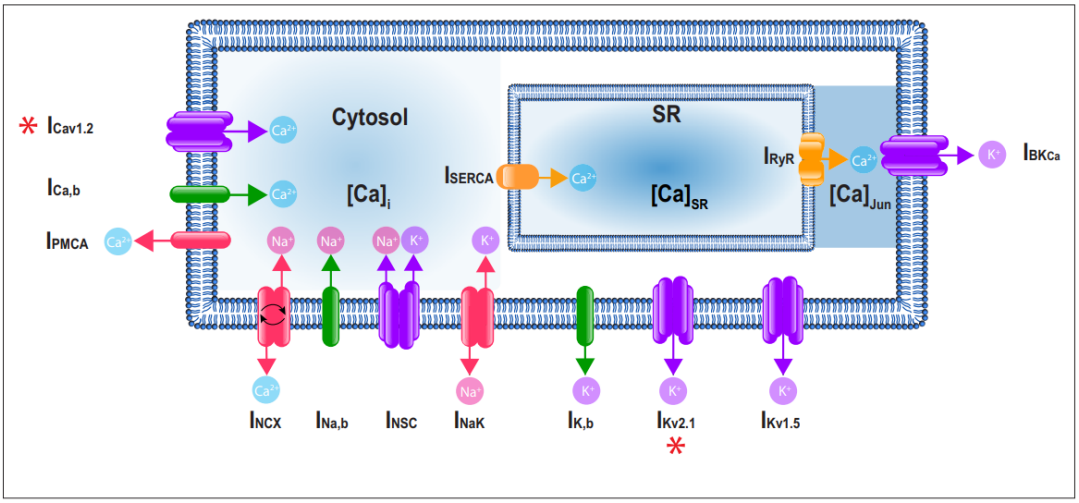


Figure 1. A schematic representation of the Hernandez–Hernandez model. The components of the model include major ion channel currents shown in purple including the voltage-gated L-type calcium current (I_{Ca}), nonselective cation current (I_{NSC}), voltage-gated potassium currents ($I_{Kv1.5}$ and $I_{Kv2.1}$), and the large-conductance Ca^{2+}-sensitive potassium current (I_{BKCa}). Currents from pumps and transporters are shown in red including the sodium/potassium pump current (I_{NaK}), sodium/calcium exchanger current (I_{NCX}), and plasma membrane ATPase current (I_{PMCA}). Leak currents are indicated in green including the sodium leak current ($I_{Na,b}$), potassium leak current ($I_{K,b}$), and calcium leak current ($I_{Ca,b}$). In addition, two currents in the sarcoplasmic reticulum are shown in orange: the sarcoplasmic reticulum Ca-ATPase current (I_{SERCA}) and ryanodine receptor current (I_{RyR}). Calcium compartments comprise three discrete regions including cytosol ($[Ca]_i$), sarcoplasmic reticulum ($[Ca]_{SR}$), and the junctional region ($[Ca]_{Jun}$). Red stars (*) indicate measured sex-specific differences in ionic currents.

Obtained from G. Hernandez-Hernandez et al., "A computational model predicts sex-specific responses to calcium channel blockers in mammalian mesenteric vascular smooth muscle," eLife, vol. 12, p. RP90604, Feb. 2024, doi: 10.7554/eLife.90604.

3 Cell Modelling With Bond Graph Tutorial

1. Navigate to the `<path_to_trainingschool01>/20250320-day03/smc_with_piezo/` directory. We will now call this directory `{CA_user_dir}` and the Circulatory Autogen directory `{project_dir}`.

   - Reminder: The `{CA_user_dir}` contains input files that define the modules that are used in your models and their parameters (and later the information for parameter identification).

   Open `{project_dir}/user_run_files/user_inputs.yaml` and set

   ```
   user_inputs_path_override: {CA_user_dir}/smc_with_piezo_user_inputs.yaml
   ```

   This model is the smooth muscle cell (SMC) model by Hernandez-Hernandez shown below, which we have replicated in CellML.

2. Navigate to `{project_dir}/user_run_files/` in a terminal and execute the following command to generate the model:

   ```
   ./run_autogeneration.sh
   ```

   The model will be generated in

   ```
   {CA_user_dir}/generated_models/smc_with_piezo/
   ```

3. Open the generated model (`smc_with_piezo.cellml`) in OpenCOR and plot the membrane potential, V_m. Increase the sodium leak channel conductance (G_{Na_b}) by 10 and 100 times to investigate how membrane voltage is affected by increasing leak sodium current.

4. Now, we are going to include a Piezo ion channel to allow us to model the response to stretch.

   The equations for current and gating of the Piezo channel we will model are:

$$I_{piezo} = g \cdot m_V \cdot m_r (V_m - E_{Na})$$

$$\frac{\mathrm{d}m_V}{\mathrm{d}t} = \frac{m_V^\infty - m_V}{\tau_V}$$

$$m_V^\infty = \frac{1}{1 + e^{-(V_m - V_{50})/k_V}}$$

$$\frac{\mathrm{d}m_r}{\mathrm{d}t} = \frac{m_r^\infty - m_r}{\tau_r}$$

$$m_r^\infty = \frac{1}{1 + e^{-(r_{ratio} - r_{50})/k_r}}$$

   where g is the maximum conductance, m_V is the voltage gating parameter, m_r is the stretch gating parameter, E_{Na} is the sodium Nernst potential, r_{ratio} is the ratio of SMC loaded length to unstressed length, and $V_m = (V_i - V_e)$.

   Draw the BG diagram for the Piezo ion channel (where the stretch gating variable m_r can be an external input) and how it connects to the rest of the model, departing from the schematic in Figure 4.

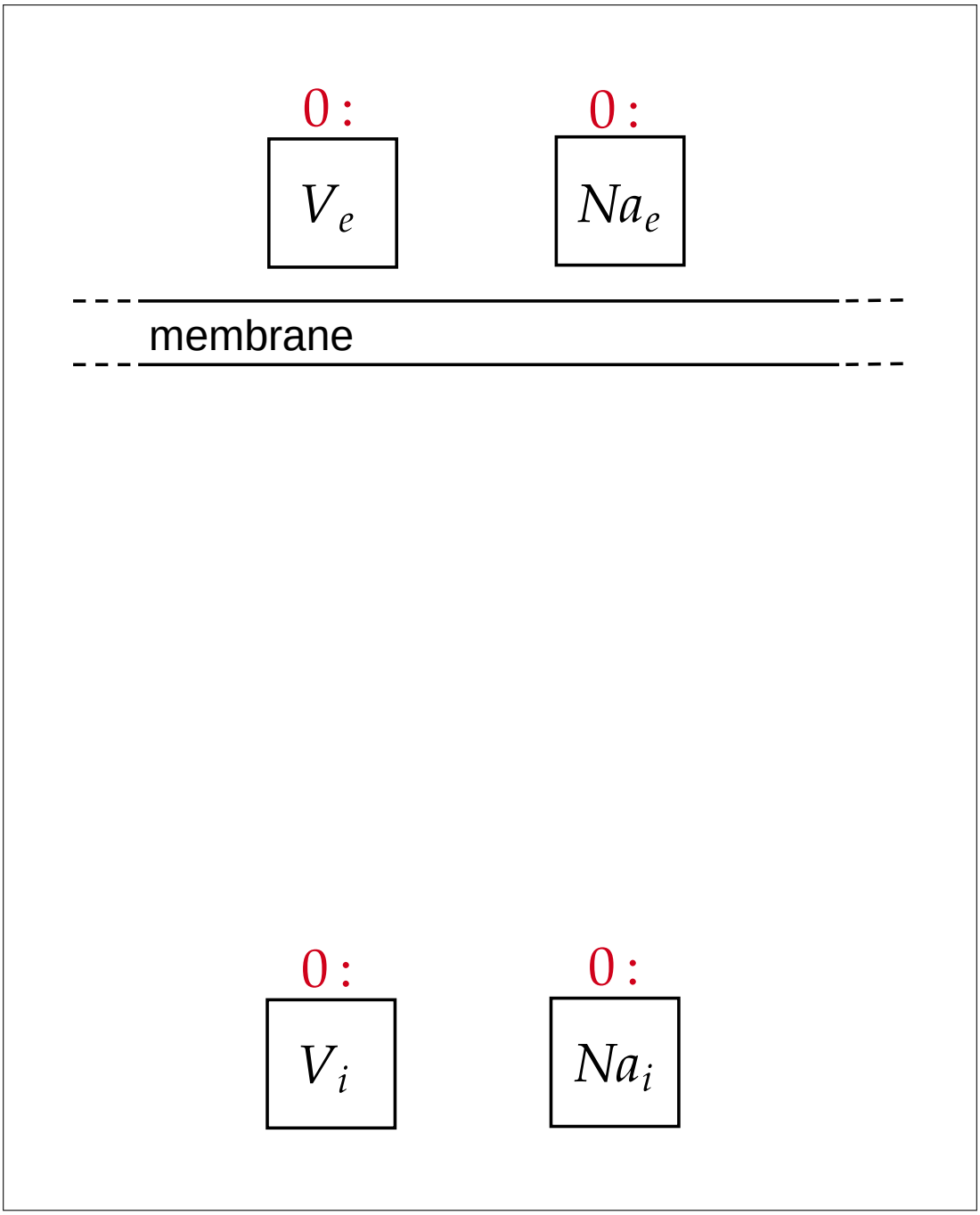


Figure 4: Start of the Piezo BG diagram.

5. Derive the conservation of charge equations for the Piezo channel and all of the other currents I_{other}.

6. Derive the conservation of molar amount equations for the Piezo channel and all other Na currents $I_{Na_{other}}$.

7. Open the file `{project_dir}/module_config_user/tutorial_modules.cellml` within OpenCOR and edit the module named `i_piezo_Na_tutorial` to include the stretch sensitive equations for the Piezo channel.

8. Open the file `{project_dir}/module_config_user/tutorial_module_config.json`, and edit the module `I_piezo_Na_tutorial` to have (i) the stretch sensitive parameters from the CellML model in `"variables_and_units"` and (ii) a general port for I_{piezo}. Since we want r_{ratio} to be set as an input in this case but to be calculated in other modules in the future, give it a type of `"boundary_condition"`.

   Copy the naming convention of the corresponding port in the `smc_hernandez_with_piezo`.

9. Open the file `{project_dir}/module_config_user/tutorial_modules.cellml` again in OpenCOR and edit

the module named `smc_hernandez_with_piezo` to include the Piezo current in the membrane voltage calculation (conservation of charge) and the Na concentration calculation (conservation of molar amount).

10. Now open `{CA_user_dir}/resources/smc_with_piezo_vessel_array.csv` and add a row named `piezo_channel` with `BC_type = nn` and `module_type = i_piezo_Na_tutorial`, with corresponding connections between the two modules.

    All model parameter values are given to you in `{CA_user_dir}/resources/smc_with_piezo_parameters.csv`.

11. Finally, repeat step 2 to generate the model.

12. Open the generated model (`{Ca_user_dir}/generated_models/smc_with_piezo/smc_with_piezo.cellml`) in OpenCOR and investigate how changing the stretch ratio r_{ratio} affects the resting membrane potential V_m and the calcium concentration Ca_i.

4 Blood Flow - Cellular Coupling Tutorial

Now, we will couple the SMC model that we developed in the tutorial from Section 3 with a vessel module within our circulatory system model from Section 2, so that the aortic vessels incorporate myogenic effects.

1. Open the file `{project_dir}/module_config_user/tutorial_modules.cellml` in OpenCOR and navigate to the `material_prop_visco_myogenic_type` module. Here, we will develop a module that calculates a changing stiffness parameter, K_{tube}, dependent on the SMC Ca concentration.

   - IMPORTANT NOTE: This simple module will not take into account myofiber mechanics, vessel geometry effects (not to mention that Piezo channel effects may be mostly through endothelial cell-SMC gap junction interactions), etc. This tutorial is used to demonstrate how cell models can be coupled to vessel models in this framework. The development of physiologically accurate control of vessel constriction from SMC dynamics will be done within VITAL.

2. Add the calculation of the stretch ratio r_{ratio} in the `material_prop_visco_myogenic_type` module

$$r_{ratio} = r/r_0.$$

3. Now include the calculation of K_{Ca}^{mod} as below

$$K_{Ca}^{mod} = 1 + \frac{1}{1 + e^{-(Ca - Ca_{50})/k_{Ca}}}.$$

   This is a sigmoid model that assumes Ca increases can only increase stiffness up to a maximum of 2x. Ca_{50} sets the midpoint of the sigmoid and k_{Ca} sets the steepness of the sigmoid. $K_{Ca_{mod}}$ modifies the uncontrolled stiffness K_{tube}^0 with the following equation:

$$K_{tube} = K_{Ca}^{mod} K_{tube}^0$$

4. Open `{project_dir}/module_config_user/tutorial_module_config.json` and navigate to the `material_prop_visco_myogenic` module and add an entry in `variables_and_units` for the K_{Ca}^{mod} variable.

5. In the same file, Add a general port named `Ca_port` with Ca as the port variable to the `material_prop_visco_myogenic` module, so that K_{tube} and SMC model can couple together.

   Note: Look at the ports in `smc_hernandez_with_piezo` within `tutorial_module_config.json` to see that there is a corresponding `Ca_port`. Modules with corresponding ports are coupled with the corresponding port variables.

6. Navigate to the `<path_to_trainingschool01>/20250321-day04/coupled_CVS_smc` directory. We will now call this directory `{CA_user_dir}`.

7. As an example, we will include myogenic effects to the `ascending_aort_A` vessel. Open `coupled_CVS_smc_vessel_array.csv` and make the `vessel_type` of `K_tube_ascending_aorta_A` equal to `material_prop_visco_myogenic`.

8. Now the modules need to be correctly connected in your desired model. Add a `smc` entry and `piezo_channel` entry to the `coupled_CVS_smc_vessel_array.csv` and make sure the `smc` is an output of `K_tube_ascending_aorta_A` and `smc` has an input of `K_tube_ascending_aorta_A` and an output of `piezo_channel`.
   We have supplied you with the parameters required for the coupled model in `coupled_CVS_smc_parameters.csv`.

9. As in previous tutorials, open `{project_dir}/user_run_files/user_inputs.yaml` and set

   ```
   user_inputs_path_override: {CA_user_dir}/coupled_CVS_smc_user_inputs.yaml
   ```

10. Navigate to `{project_dir}/user_run_files/` and execute the following command to generate the coupled model:

    ```
    ./run_autogeneration.sh
    ```

11. Open the generated model (`{Ca_user_dir}/generated_models/coupled_CVS_smc/coupled_CVS_smc.cellml`) in OpenCOR and investigate how changing the sigmoid steepness k_{Ca} (in `K_tube_ascending_aorta_A`) by an order of magnitude up and down affects the pressure u and stressed volume q_C profiles in the aortic segment `ascending_aorta_A`.